

Seminar 3

Object-Oriented Design, IV1350

Ellen Grönholm
ellengro@kth.se

VT26 v20

Declaration:

By submitting this assignment, it is hereby declared that all group members listed above have contributed to the solution. It is also declared that all project members fully understand all parts of the final solution and can explain it upon request.

It is furthermore declared that no part of the solution has been copied from any other source (except for resources presented in the Canvas page for the course IV1350), and that no part of the solution has been provided by someone not listed as a project member above.

Usage of AI

The usage of AI assistance in the report is limited to help of environmental issues with the IDE and explaining observed errors when coding. For this report, AI was also used to proofread and suggest stylistic changes to the language. As AI assistance ChatGPT-5.5 (OpenAI) and Gemini 3 Flash (Google) have been used.

1 Introduction

The objective of this seminar task is to practice translating an object-oriented design model, specifically the one developed in Seminar 2, into functional Java source code, while simultaneously establishing a robust automated testing framework. Specifically, this implementation focuses on executing the basic flow and startup sequence for an Electric Bike Repair Workshop system.

The scope of this seminar is strictly bounded to the primary success scenario; alternative flows (such as handling an unknown phone number) and robust error handling via exceptions are intentionally omitted, as they are reserved for Seminar 4. Through this process, core Object-Oriented Design principles, Java coding conventions, and systematic unit testing using JUnit are applied and evaluated.

2 Method

2.1 Programming Workflow and Implementation Choices

The workflow prioritized a systematic translation of the Seminar 2 Multi-Layered Architecture (View, Controller, Model, and Integration layers) into clean, maintainable Java code.

- **Layer Separation:** Code was strictly modularized to maintain clear boundaries. The **View** interacts solely with the **Controller**, which orchestrates calls to the **Model** and **Integration** components.
- **Encapsulation & Data Transfer:** To prevent the **View** layer from gaining deep, unintended structural access to the model, Data Transfer Objects (DTOs) were designed and implemented. State is passed to the **View** exclusively via immutable DTOs rather than returning direct references to mutable domain objects.
- **Best Practices:** Industry-standard Java naming conventions were followed. Code readability was prioritized by maintaining short, single-purpose methods. Documentation strictly adheres to the rule of providing exactly one Javadoc comment per public declaration, intentionally avoiding redundant or inline comments inside method bodies.

The implementation process began by revisiting the design created in Seminar 2. The UML diagrams were used as a foundation. According to the lecturer, the UML diagrams did not have to be updated, but adjustments were made throughout the project as it helped visualizing my thoughts. All in all the UML was changed a bit, especially during the flow of the problem description as it became clear coupling could be lowered.

Another aspect that changed was how the view communicated with the controller, as it is now deemed that the view might send multiple messages depending on user input

that earlier was not understood how it worked. The view is still not fully implemented as a view, instead it is fully hard coded as method-*anrop*. To get this understanding, the issue was discussed with classmates who described how the view usually communicates with the controller whilst the author explained how they had imagined the view interface.

The development process was carried out incrementally:

1. Creating the overall project structure and packages.
2. Implementing core classes for the main flow.
3. Adding a simplified user interface using a single **View** class with hard-coded method calls.
4. Following the separate flows to ensure all aspects were covered.
5. Writing tests for the newly done flow

The points 4-5 were repeated for all flows or communication diagrams. By separating it based on the flows, it was easier to limit the thoughts to only one aspect at the time while also giving a direction of where to start.

Unit tests were written using the testing frameworks Maven and JUnit 5. The focus was on testing core functionality but when possible trivial methods were also tested. In none of these cases were alternative flows tested as that is to be done for next Seminar.

During the implementation, the greatest headache was the environment. While the author has done programming before, the usage of packages, projects and especially Unit-testing was new. A lot of hours were thus spent on getting the environment to work with the author as these problems always seemed to arrive at the worst times.

The Unit testing was also new, and Maven was installed for this to work. For the functioning of the environment, AI (ChatGPT 5, OpenAI; Gemini, Google; Claude Code; Claude AI) were used. But to stay as vigilant as possible, while also adapting to the new world, the AI were cross-referenced and the solutions asked to explain what was done. While the author might not still understand it 100%, it is now understood well enough that it can be explained what is done, and that she might be able to help others in the future.

The Object **Task** might not be implemented well as it needs multiple methods to sent three parameters forward until a **Task** is created. It was debated if a **TaskDTO** should have been created, or if the controller should have accessed the **Task** directly. In the end it was left as is. Feedback is happily received.

***Task** is the object that is listed in the proposed repairs. The three parameters must thus be sent from the view, through the controller, through the Repair Order, and lastly to the Recommended repairs where the object Task is created.*

Lastly, an assumption is made that all actors have access to the Repair Order ID. It might be written on a post it and put on the bike. The customer might also get a written post-it note with it. However the view can input the Repair Order ID without storing it somewhere.

2.2 Testing Strategy

To ensure code quality, a comprehensive unit testing strategy was executed using the JUnit framework.

- **Scope of Coverage:** Automated unit tests were written for every class across the layers. The only exclusions permitted were classes containing exclusively simple getters, setters, or basic constructors with no conditional logic.
- **Test Case Selection:** Test cases were selected by imagining all possible situations and outcomes. Special attention was paid to conditional logic to ensure that all branches of any internal `if`-statements were fully executed. If some code worked incorrectly, new tests were added instead of running the code with different parameters.
- **State Evaluation:** Tests were designed to be completely self-evaluating using descriptive JUnit assertions (`assertEquals`, `assertTrue`, etc.), eliminating any need for manual console printout verification during execution.

When writing the tests, all aspects where code might fail were taken into consideration except for handling alternative flows. Most tests used either `assertEquals` or `assertNotNull`. Two tests were also left as `@Disabled` one as it was handling alternative flows, and one as the author did not know how to test it without including methods in the SUT.

Throughout the code, comments with method-names were left to act as reminders for more tests that can be added.

Other than that, all test that crossed the authors mind were added, as there can not be too many tests.

3 Result

3.1 Program Overview

The implemented system successfully executes the basic startup and primary flow of the Electric Bike Repair Workshop.

- **Execution Flow:** The `Main` class boots the system, instantiates the integration layer components (simulating database storage via in-memory collections within objects), passes them to the `Controller`, and starts the `View`.

- **View Simulation:** Because a user interface is not integrated, the **View** class acts as a test driver, executing a hard-coded sequence of method calls to the **Controller** representing a standard repair registration scenario. All returned DTO values and status updates are printed directly to **System.out**.

3.1.1 System Behavior Specifications

The code is implemented to follow the structure of the interaction diagrams. The flow between classes is thus taken from there and implemented. The main flows include the startup of the system, finding the customer, adding a problem list (which created the repair order), adding a diagnostic report, adding the proposed repairs, summarizing the repair order and wait for the deciding status and finally printing a summary.

3.1.2 Sample run

Figure 1 illustrates the overall flow and the output then. Not much is returned other than the customer so it can be sent back when creating the Repair Order and then the Repair Order ID, which the receptionist must give to the customer and the mechanic.


```

=====Reciept=====
Customer Info:
-----
Name:          ellen
Phone Number:  0733976999
Email Address: ellen.gronholm@hotmail.com
Bike:
      SerialNumber: 23490
      Model:        dam
      Brand:        helkama

Problem Description:
-----
Current bike faults:
- TIRE
- BRAKES

Diagnostic Report:
-----
Current bike faults:
- TIRE
- BRAKES

Propossed Repairs:
-----
Current recommended bike repairs:
- Replace tire:      200
- Fix chain:    150
      total:          350

=====
Date:          Mon May 25 16:06:00 CEST 2026

ID:            0
=====

```

Figure 2: The reciept created to a file to be printed.

3.2 Testing Overview

Unit tests were implemented to verify functionality. The tests focus on validating business logic and interactions between components. All test classes contained a `@BeforeEach` and `@AfterEach` and multiple `@Test`:s. The tests follow in general the structure of adding an object to a repository and then manipulating it or double-checking that everything works as it should.

In some cases it was argued if these Objects shouldn't be created in the `@BeforeEach`, but since they were not used for all methods, it was not done.

The tests confirmed that the system behaves correctly under expected conditions.

A total of 56 unit tests were implemented. The test suite achieves comprehensive coverage across the layers. `@assertTrue`, `@assertNotNull`, and `@assertEquals` were the most used test forms if the test code.

3.3 Git Repository

Public Git Repository: <https://gits-15.sys.kth.se/ellengro/IV1350-ObjektOrienteradDesi>

Note that the Git repository also contains all printouts and files created by the code.

4 Discussion

4.1 Code Quality Assessment

The code is hopefully easily read and understood. Since aspects can be obvious for the developer that has thought about everything, others without the in dept knowledge might not understand the code in the same way. The goal was that the finalized source code demonstrates strong adherence to professional clean code standards:

- **Naming and Readability:** Class, method, and variable names reflect the domain language of the Electric Bike Repair Workshop. Furthermore all naming follows java standards and try to be descriptive and standardized through the code and between classes when possible.
- **Code Duplication:** Redundancy was actively neutralized. Common code blocks (such as repetitive lookups or formatting) were refactored into private helper methods that could be called by multiple other methods. If relevant, methods were made package private as this was deemed better than duplicate code.
- **Sizing Rules:** Classes and methods were kept short and relevant to the method name. This is to ensure that each function performs one task.

4.2 Object-Oriented Design Analysis

The transition from the design model to implementation provided excellent insights into the UML process as well and gave knowledge that will better future projects. It also forced choices to be made about the structuring of the code:

- **Primitives vs. Objects:** The system minimizes primitive obsession. As many as possible specific metrics are wrapped inside dedicated objects or enums rather than raw `int` or `String` variables.
- **Cohesion and Coupling:** High cohesion was achieved by assigning highly related responsibilities to dedicated classes (e.g., printing creation logic resides entirely within the `printsystem` layer). Coupling was kept low by relying on the layer structure using packages. The data for example can only be accessed through the `dbhandler`.

- **Encapsulation & DTO Implementation:** Encapsulation was rigorously maintained. Domain classes do not expose their internal state collections directly. By converting domain model entities into immutable DTOs before passing them up to the **View**, the view layer is strictly blocked from gaining "the direct possibility to get to everything" or modifying internal business states out-of-band.

4.3 Evaluation of the Test Suite

The JUnit implementation confirms that the software is robust and maintainable:

- **Self-Evaluation:** Every test is fully autonomous. Assertions compare actual method outputs against expected values directly in memory, requiring zero human intervention or visual console tracking to determine a pass/fail state.
- **Branch Coverage:** By testing both typical inputs and boundary conditions, all logical pathways inside operational conditional statements (**if/else**) were executed and validated, as well as other scenarios that could go wrong. For cases that might be implemented for the alternative flow, comments were left in the test classes to remember to add those flows in the future.
- **Interesting Parameter Values:** Test cases avoided generic data; instead, they leveraged diverse and complex parameters (e.g., zero-values, extreme pricing bounds, and non-sequential IDs) to ensure the system handles edge cases gracefully within the context of the basic flow.

To this all print to file functions were tested, and the print to file function was used to test other output by printing it to a file and then reading it from said file.

5 References

In general this report is based on knowledge received from the course IV1350, the discussions it has encouraged and the material in its canvas room.

- GeeksforGeeks. (2025, July 11). Java current date and time. GeeksforGeeks.
<https://www.geeksforgeeks.org/java/java-current-date-time/>
- GeeksforGeeks. (2026, March 13). File handling in Java. GeeksforGeeks.
<https://www.geeksforgeeks.org/java/file-handling-in-java/>
- Lindbäck, L. (March 23 2026) *A First Course in Object-Oriented Development: A Hands-On Approach*.
- Paraschiv, E. (2023, December 1). Java – Write to File. Baeldung.
<https://www.baeldung.com/java-write-to-file>

6 Re-submission

The code has had a few changes since the last submission:

- the main method does not connect to the controller anymore as that was requested. Instead the controller is triggered by the view instead of being called directly from main.
- The methods in controller earlier returned a void. After speaking to Niharika it was decided to return repair Order DTO's instead, which is the custom now. Although this is not optimal for the hardcoded view which must print all returns to the terminal, it is of good use when adding the view as it can return the updated lists to the view from the DTO's.
- The print function has been bettered as separated to the classes `PrintToString` and `PrintToReciept` as these two logics did not need to be in the same class.

Other than that, parts of the method have been rewritten and the discussion has been restructured.